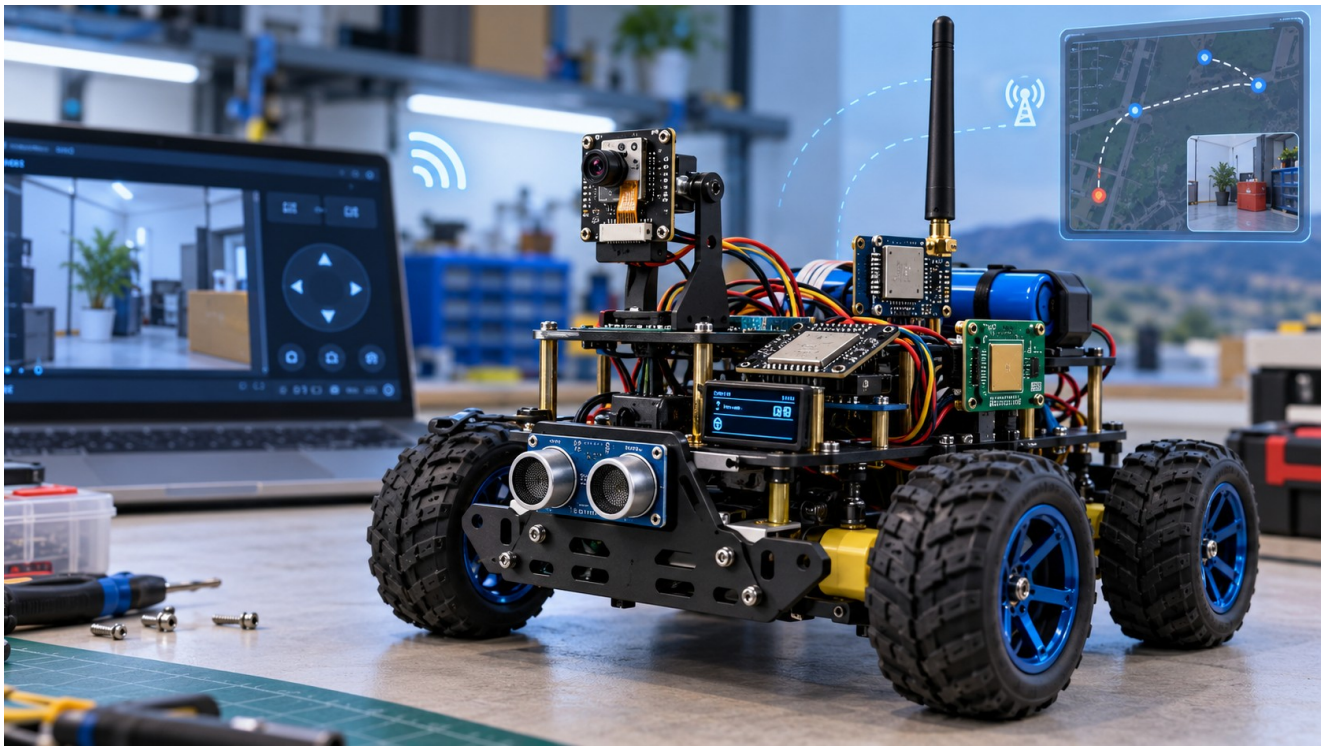
AURA-X

AI-POWERED EXPLORATION ROVER

Manual control | Autonomous obstacle avoidance | Camera vision | LoRa telemetry



Concept visualization generated for this report. It is illustrative, not a verified photograph or finished hardware layout.

PROJECT THESIS

AURA-X combines a locally controlled mobile rover, an ESP32-CAM vision unit, a LoRa command/telemetry link, and an AI gateway. The AI proposes high-level actions; the rover firmware retains direct authority over motors and safety.

REFERENCE REPORT • HARDWARE + SOFTWARE + BUILD ROADMAP

Prepared for project development and presentation planning | 14 October 2026

1. Executive Summary

AURA-X is a multi-mode exploration rover intended for indoor test environments and controlled outdoor demonstrations. It combines manual teleoperation, sensor-based autonomous obstacle avoidance, camera-based scene understanding, and long-range low-bandwidth communication. The design is deliberately split across a main ESP32 controller, an ESP32-CAM vision module, and a remote station containing a matching LoRa radio.

The system uses local firmware for time-critical behaviours such as motor PWM, obstacle checks, command timeout, and emergency stopping. AI is used for tasks that benefit from high-level reasoning: interpreting a natural-language mission, classifying a camera snapshot, summarizing rover status, and recommending a safe next action. It must not be the only thing preventing a collision.

Project objective

- Build a rover that can be driven manually and can perform a repeatable local obstacle-avoidance routine.
- Send compact commands, status, and sensor events over a LoRa link independent of Wi-Fi video.
- Stream camera imagery over Wi-Fi when coverage exists and submit selected frames to a vision-capable AI API through a laptop gateway.
- Provide a dashboard with mode, video, sensor readings, link status, battery status, mission log, and stop controls.
- Demonstrate safe degraded behaviour when the AI API, Wi-Fi camera path, or LoRa link becomes unavailable.

Recommended first demonstration

THE 90-SECOND DEMO

Drive manually from the dashboard, switch to autonomous mode, let the rover stop and turn around an obstacle, request an AI inspection of a captured frame, then disable Wi-Fi and show that LoRa telemetry and local stopping still work.

Scope boundary

The first prototype is not a self-driving mapping robot. Without a depth camera, lidar, wheel odometry and localization, it will not reliably build a metric map or know its exact location. The first goal is dependable reactive navigation plus AI-assisted inspection. Mapping, advanced path planning and onboard computer vision are later upgrades.

2. Requirements and Feature Scope

Functional requirements

ID	Requirement	Acceptance intent
FR-01	Manual drive	Forward/reverse/left/right, adjustable speed, immediate operator stop.
FR-02	Video	ESP32-CAM provides a low-resolution stream or periodic JPEG snapshots over Wi-Fi.
FR-03	Local obstacle avoidance	Ultrasonic readings gate forward motion; blocked path causes stop, scan/turn, then re-measure.
FR-04	LoRa telemetry	Send heartbeat, mode, sensor summary, battery voltage (if measured), fault code and event counter.
FR-05	LoRa commands	Receive whitelisted commands with sequence numbers, acknowledgements and expiration.

ID	Requirement	Acceptance intent
FR-06	AI missions	Convert natural language into a limited mission schema; AI visual inspection operates on selected images.
FR-07	Fail-safe	On stop command, stale control lease, critical fault or emergency-stop activation, disable motor drive.
FR-08	Event logging	Store timestamp/sequence, event type, sensor evidence and action taken on the gateway.

Non-functional requirements

- Modular firmware: motor, sensing, radio, camera, UI and AI gateway code are separate modules.
- No cloud dependency for emergency stop or elementary obstacle avoidance.
- No long or blocking API call inside the real-time motor-control loop.
- Defined fallback behaviour for sensor failure, communication loss, brownout and watchdog reset.
- All unverified claims (range, battery runtime, detection accuracy) must be measured rather than assumed.

Build target levels

Level	What is included	Status target
MVP	Manual drive, motors, ultrasonic safety stop, LoRa command/telemetry, physical stop.	Must be stable first.
Core demo	Camera stream, dashboard, autonomous obstacle routine, IMU telemetry.	Main demonstration.
AI upgrade	Mission JSON, image interpretation, AI event summary, command validation.	Only after core is stable.
Future	Encoders, mapping, GPS outdoors, multiple camera angles, better onboard CV.	Not required for first build.

3. System Architecture

The following diagram shows the recommended logical architecture. Keep the main ESP32 responsible for driving and safety. The remote computer handles the user interface and AI API calls. The camera has its own Wi-Fi image path; the LoRa radio carries small structured packets, not video.

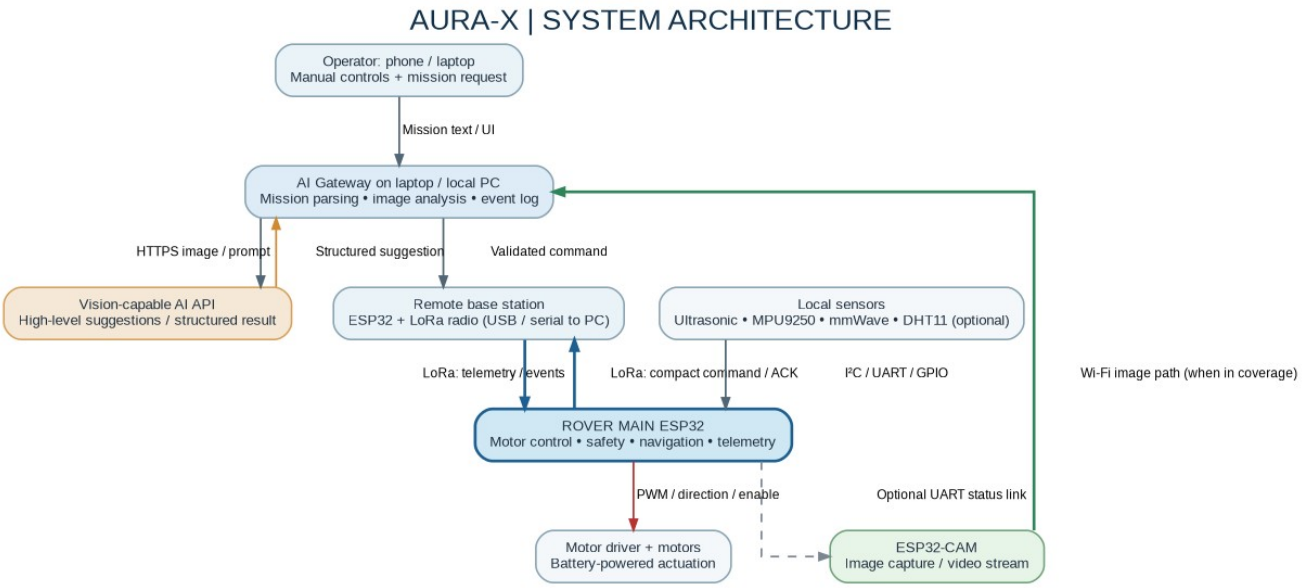


Figure 1. Logical architecture. The exact camera-to-gateway transport depends on the chosen ESP32-CAM firmware and Wi-Fi topology.

Subsystem responsibilities

Subsystem	Owns	Must not own
Main ESP32	Motor PWM/direction, local obstacle logic, command validation, mode state, sensor polling, LoRa packet handling.	Cloud AI calls or long waits that block motor updates.
ESP32-CAM	Image capture, camera stream/snapshots, optionally a minimal health/status interface.	Primary motor safety logic.
Remote base	Matching LoRa radio, serial bridge, link stats and operator-side packet handling.	Sending unchecked raw motor PWM from AI text.
Laptop gateway	Dashboard, event log, API key storage, selected-frame upload, mission parser and command allowlist.	Bypassing rover-side safety rules.
Sensors and driver	Provide measurements and actuate motors within their electrical ratings.	Being powered directly from a GPIO pin.

ARCHITECTURE RULE

The main ESP32 is the final authority on whether a movement command is permitted. The AI gateway sends intent; the rover firmware enforces limits.

4. Communication Design: Wi-Fi + LoRa

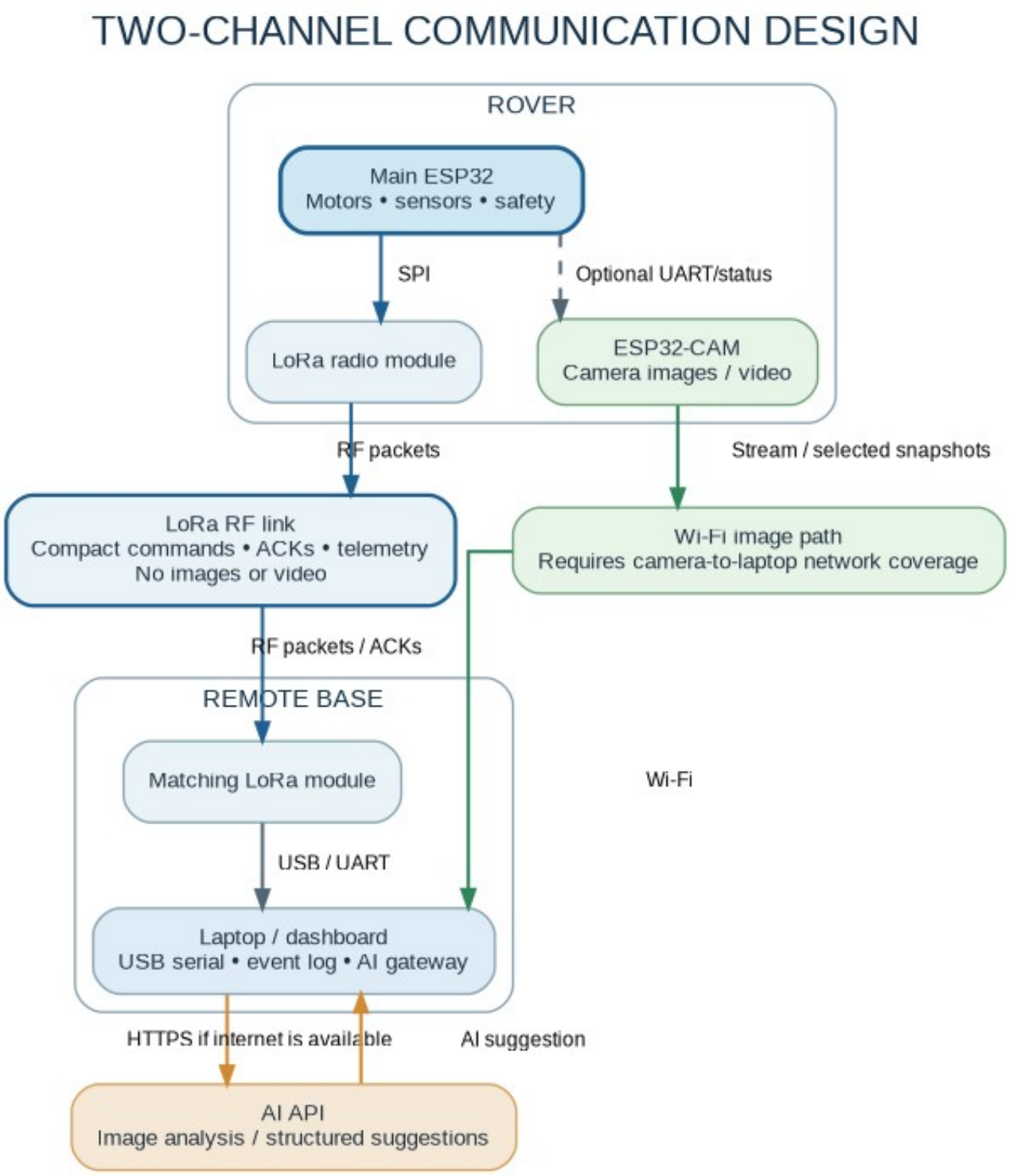


Figure 2. Split communication channels. Wi-Fi is for high-bandwidth image transfer; LoRa is for compact command and telemetry packets.

Use LoRa for

- Mission start/cancel and low-rate manual movement commands.
- Heartbeat, link quality, sensor summaries, battery/status reports, fault codes and event acknowledgements.
- A stop request as a secondary path; local physical stop and firmware timeout remain primary safety mechanisms.

Do not use LoRa for

- Live video, raw high-resolution images, continuous IMU streams, or frequent joystick updates that assume Wi-Fi-like bandwidth.

Suggested link protocol

Field	Purpose	Implementation guidance
Protocol version	Allows future packet-format changes.	Start with version 1.
Message type	CMD, ACK, HEARTBEAT, TELEMETRY, EVENT, STOP.	Whitelist types; reject unknown values.
Node ID	Distinguishes rover and base.	Unique fixed ID per device.
Sequence number	Detects duplicates and stale packets.	Increment every new command.
Command + parameters	Small enumerated action plus speed/duration.	No arbitrary code or raw AI-generated pin values.
Expiry / lease	Limits how long a movement command remains valid.	Use a short calibrated lease; stop when it expires.
Integrity / authentication	Detects corrupt packets and unauthorized commands.	Radio CRC is not authentication; add application-level authentication for later versions.
ACK + retry	Confirms receipt and helps characterize packet loss.	Bound retries; do not block the motor task.

Frequency and module compatibility

Both LoRa radios must use compatible hardware, frequency band, bandwidth, spreading factor, coding rate, sync word and packet settings. “SX1278/RA-02” is a common 433 MHz-labelled module family, but do not assume the exact frequency from appearance alone: read the module marking and datasheet. India’s WPC/DoT rules apply to the relevant RF equipment and bands; verify the permitted band, power, antenna and applicable approval requirements before field use. See references [4] and [7].

5. Sensors and Hardware Allocation

Component	Role	Integration notes	Priority
ESP32 development board	Main controller	Motor control, local navigation, UART/I²C/GPIO, LoRa driver and Wi-Fi if needed.	Essential
ESP32-CAM	Vision node	Image stream or selected JPEG snapshots. Confirm exact board variant and camera sensor.	Essential for vision
LoRa modules x 2	Rover + remote base	SPI radio connection at rover and base. Exact model/frequency must match.	Essential for long-range link
Motor driver	Actuator interface	Choose based on motor supply voltage and each motor’s stall current. Four motors may exceed a small driver’s current limit.	Essential
DC geared motors + chassis	Mobility	A 2WD or 4WD chassis; use matching motors and wheel sizes.	Essential
HC-SR04 ultrasonic	Near-field range	Use for simple forward obstacle distance. Level-shift/divide ECHO to 3.3 V before an ESP32 input.	Essential
MPU9250 / MPU6050	IMU	Angular rate/acceleration for orientation and motion estimates. Not absolute positioning.	Recommended
mmWave radar	Presence/motion event	Useful for human presence if module supports it; not a generic obstacle-distance sensor. Model-specific UART/GPIO interface.	Optional upgrade
DHT11	Environment	Ambient temperature/humidity display; not important to navigation.	Optional
Wheel encoders	Odometry	Estimate wheel movement and reduce open-loop motion	Good upgrade

Component	Role	Integration notes	Priority
		uncertainty.	
Battery, fuse, switch, buck converter	Power system	Separate motor rail from regulated logic rails; common ground where interfaces connect.	Essential
Physical emergency stop	Safety	Hardwire to disable motor-driver enable or interrupt the motor supply.	Strongly recommended
Buzzer / status LEDs / OLED	Local feedback	Operator feedback even when Wi-Fi/API is down.	Recommended

Which parts to leave out of version 1

Skip DHT11, GPS, a servo scan head and advanced mapping until the motor/radio/safety baseline works. A single forward ultrasonic sensor is enough to prove the first avoidance loop; later add a servo to scan left/right or use a second/third fixed range sensor if suitable.

mmWAVE CAUTION

Identify the exact radar model before writing its interface. An LD2410-style presence sensor can indicate human presence/motion in its field of view; it does not give a complete 2D obstacle map and should not replace ultrasonic range sensing.

6. Power and Wiring Reference

Do not finalize GPIO pin numbers until the exact boards and modules are on the bench. ESP32 variants differ, and the ESP32-CAM has camera, flash and boot-strapping pin constraints. Use the manufacturer documentation for the specific board and validate the boot behaviour after wiring. See [1] and [2].

Power layout

Power branch	Supplies	Key rules
Battery / motor rail	Motor driver VM and motors.	Check motor rated voltage and stall current; add an accessible switch and appropriately rated fuse. Avoid powering motors from the ESP32 5 V/3.3 V pin.
Regulated 5 V rail	ESP32-CAM (if the chosen board requires 5 V input), main board VIN/5 V if supported.	Regulator must tolerate camera/Wi-Fi peaks and board requirements. Do not assume every ESP32 board accepts 5 V at the same pin.
3.3 V logic rail	LoRa module, logic-level sensors, IMU where module supports 3.3 V.	Check the actual breakout's supply range and peak current. RA-02/SX127x radio modules are 3.3 V logic; never connect them to 5 V GPIO.
Ground reference	Logic modules and motor driver logic where required.	Use a common ground for non-isolated signal interfaces; route high motor current away from sensitive sensor/radio wiring.

Interface-level connection plan (not final GPIO pinout)

Device	Interface	Signals / notes
LoRa transceiver	SPI + IRQ / reset	SCK, MOSI, MISO, NSS/CS, RESET and DIO0 (exact pins library/module dependent). 3.3 V logic only.
MPU9250	I^2C or SPI	SDA/SCL for I^2C; verify address and breakout voltage support. Keep wiring short.
HC-SR04	GPIO timing	TRIG from controller. ECHO is commonly 5 V on standard modules; use a resistor divider or level shifter to protect 3.3 V input.

Device	Interface	Signals / notes
mmWave	UART / GPIO / vendor interface	Use model-specific pinout and voltage. Don't assume every mmWave board has the same TX/RX behaviour.
DHT11	Single-wire digital	Optional; use correct pull-up arrangement for the breakout and supply.
Motor driver	PWM + direction + enable	Driver logic pins from controller; motors powered from suitable motor rail. Verify driver current rating at startup/stall.
ESP32-CAM	Wi-Fi / optional UART	Use Wi-Fi for camera stream when possible. If UART is used, reserve available pins carefully and define a small status protocol.

Before first power-on

- Check polarity and continuity with battery disconnected.
- Confirm no 5 V signal can reach an ESP32 GPIO.
- Power the logic rail without motors and check that both MCUs boot.
- Test the driver with wheels lifted off the surface and speed limited.
- Press the physical stop and confirm the motor driver is actually disabled.
- Measure voltage under motor startup; reset loops often indicate a weak supply or poor grounding.

7. AI Mission Control and Vision Pipeline

The AI component is a high-level planner and image interpreter, not the low-level motor controller. A laptop/local PC gateway keeps API credentials off the ESP32 and allows easier logging, image resizing and validation. A vision-capable API can analyze selected camera frames; see [5]. A constrained structured-output schema helps the gateway parse a response, but schema compliance alone does not guarantee a safe or correct recommendation; see [6].

1. Operator enters a mission, such as “Explore the area and look for a red object.”
2. Gateway validates the request and turns it into a small mission object.
3. Rover runs local obstacle checks continuously and sends sensor summaries/events.
4. When image analysis is useful and the camera frame reaches the gateway, the gateway submits a resized frame and a focused question to the vision API.
5. AI returns a constrained suggestion such as TARGET_SEEN, CONTINUE_EXPLORE, TURN_LEFT, TURN_RIGHT or STOP_AND_REPORT.
6. Gateway and rover independently validate the suggestion against the allowlist, current mode, obstacle limits, command lease and stop state.
7. Rover performs only a bounded action, gathers new readings, and repeats. Events are logged with timestamps / sequence IDs.

Example high-level mission JSON

```
{
  "mission_id": "M-001",
  "goal": "find_object",
  "target_description": "red object",
  "speed_profile": "slow",
  "max_duration_s": 30,
  "on_target": "stop_and_report",
  "on_link_loss": "safe_stop"
}
```

AI command allowlist

Allowed action	Meaning	Rover-side checks
STOP	Disable motors and end current action.	Always accepted; implement locally too.
EXPLORE	Start local reactive navigation under duration limit.	Auto mode enabled, all sensors sane, battery okay.
CAPTURE_IMAGE	Request / await a camera frame.	Camera available; no motor movement required.
TURN_LEFT / TURN_RIGHT	Perform short bounded turn.	No safety fault; calibrated duration; then re-read sensors.
MOVE_FORWARD	Drive a short timed segment.	Front clearance above threshold; speed and duration capped.
REPORT_STATUS	Return telemetry / event summary.	No movement.
END_MISSION	Stop and mark mission complete/cancelled.	Stop motors and log reason.

SECURITY AND PRIVACY

Never embed an API key in firmware, mobile HTML, or a public repository. Keep it on the gateway or backend, limit uploaded images, avoid retaining unnecessary footage, and do not treat AI vision output as proof of identity or a safety certification.

8. Navigation and Safety State Machine

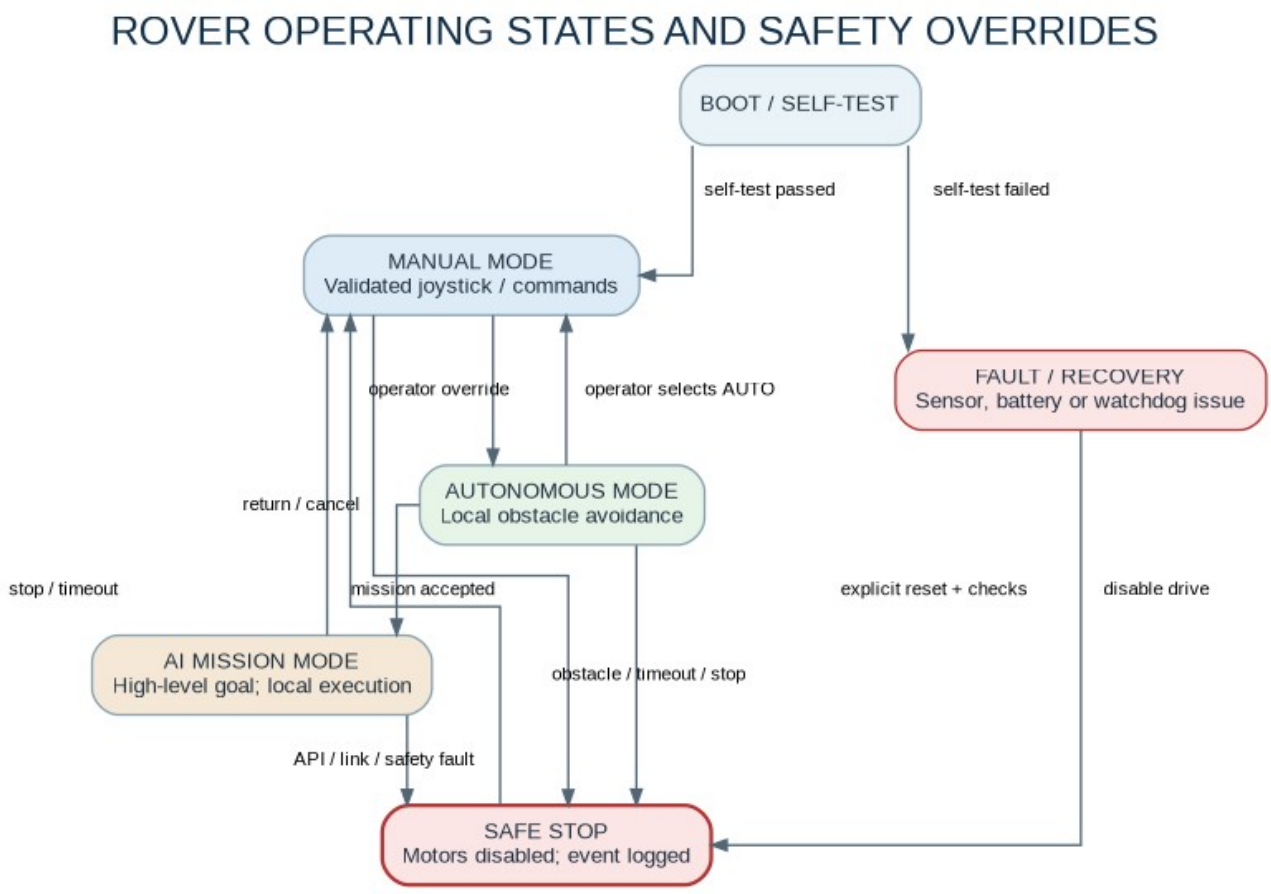


Figure 3. Illustrative state machine. Any active mode can transition to SAFE STOP on a critical safety condition.

Recommended navigation loop

1. Read range sensor; reject missing, stale or implausible measurements.
2. If emergency stop is pressed, sensor fault is critical, command lease expires, or a safety distance is violated: disable drive immediately and log the cause.
3. In manual mode, accept only bounded speed/direction commands from the selected trusted control channel.
4. In autonomous mode, move a short distance, stop to measure, then choose a clear direction; avoid long blind forward segments.
5. In AI mission mode, execute local navigation between high-level inspection checkpoints. Do not wait for an API call while motors are moving.
6. On target detection or mission timeout, stop, capture a report and await next operator instruction.

Initial safety parameters to tune during testing

Parameter	Starting design approach	How to calibrate
Obstacle stop distance	Begin conservatively (for example, 25–35 cm for a small slow rover).	Test at multiple approach speeds and with different surfaces/angles; increase margin if braking distance is inconsistent.
Command freshness	Every movement command has a short lease; stale manual drive commands expire.	Start around 0.5–1.5 s for a bench prototype, then measure link behaviour and ensure fail-safe stop is timely.
Maximum action duration	No unbounded move command; use short motion primitives.	Choose limits based on measured speed and test-area dimensions.
Link-loss behavior	Stop motors; keep sensing and attempt bounded reconnection.	Test loss of LoRa, Wi-Fi, and laptop/API separately.
Battery low threshold	Measure battery voltage through an appropriate divider/ADC or use a suitable monitor.	Set threshold for the actual chemistry and cell count; do not guess.
AI timeout	Treat unavailable/slow API as no new suggestion.	Use a timeout/cancel path; local control must remain responsive.

The numerical examples above are initial bench-test parameters, not universally safe values. The appropriate distances and timeouts depend on speed, mass, traction, sensor field of view, radio configuration and the environment.

9. Firmware and Software Structure

Rover firmware modules

Module	Responsibilities	Typical execution pattern
motor_control	Set PWM/direction, ramp speed, motor enable, emergency stop output.	Fast periodic updates; no blocking I/O.
safety_supervisor	State machine, stale-command timer, obstacle guard, fault latch, recovery conditions.	Highest-priority logic; can veto any requested movement.
sensor_manager	Ultrasonic timing, IMU reads, mmWave events, optional DHT11.	Periodic / asynchronous reads with validity flags.
navigation	Short motion primitives, obstacle response, optional heading hold.	State-based; always interrupted by safety supervisor.
lora_link	Packet encode/decode, node IDs, ACK, retry count, sequence freshness, link stats.	Non-blocking receive and bounded retries.
camera_bridge	Optional UART status or camera health messages.	Separate from critical motor loop.

Module	Responsibilities	Typical execution pattern
telemetry	Publish compact state snapshots and fault/event messages.	Rate-limited; never flood LoRa.
configuration	Calibrated distances, speed caps, node ID, radio settings.	Stored with safe defaults and versioning.

Gateway software modules

- Web UI: manual controls, video panel, mode selector, telemetry, event timeline, AI mission textbox and connection state.
- LoRa serial bridge: binary or compact framed packets, ACK handling, packet counters, and connection health.
- Camera client: connect to the ESP32-CAM stream or image endpoint while on the same reachable network; capture frames at a limited rate for AI inspection.
- Mission planner: convert user language into the mission schema and reject invalid mission objects.
- AI adapter: store API key in an environment variable/server-side config; request structured action suggestions and image analysis.
- Safety policy: whitelist, rate-limit, expire and log every action before forwarding it to the rover.

Recommended implementation choices

Area	Practical starting choice	Reason
MCU firmware	Arduino framework for ESP32 or ESP-IDF if more control is needed.	Use the ecosystem you can debug quickly; keep the architecture modular.
Camera	Espressif camera example / AI-Thinker-compatible sketch once board is verified.	Start with a working stream/snapshot before integrating AI.
Gateway	Python (FastAPI/Flask) or Node.js local server.	Simple serial integration, logging and API key isolation.
Dashboard	Plain HTML/JS or lightweight web UI.	Responsive and fast to change during hackathon development.
AI output	Structured JSON schema plus independent validation.	Reduces parser ambiguity but does not replace safety checks.

10. Build Roadmap and Work Breakdown

Phase A — Bring-up (first 2–4 hours)

- Identify all exact module names and verify supply/logic voltage.
- Test each motor channel with wheels raised and a low speed cap.
- Read HC-SR04 distance and IMU data independently.
- Bring up a LoRa point-to-point link using minimal test firmware.
- Make the rover stop on a physical emergency button.

Phase B — Minimal viable rover

- Implement the mode/state machine and command freshness lease.
- Add local ultrasonic stop and turn routine.
- Create compact LoRa telemetry and acknowledgement packets.
- Test with Wi-Fi and AI completely disabled.

Phase C — Camera + dashboard

- Bring the ESP32-CAM online on the same Wi-Fi network as the laptop.
- Add a dashboard video panel plus manual controls.
- Keep camera traffic separate from the LoRa packet stream.

- Verify manual stop works if the browser refreshes or loses connection.

Phase D — AI gateway

- Send one selected snapshot to a vision-capable API and verify output.
- Add a strict schema for mission and action values.
- Add the allowlist and gateway log before connecting AI decisions to movement.
- Demonstrate AI output as advisory first; enable bounded actions only after tests pass.

Phase E — Final integration

- Test LoRa loss, Wi-Fi loss and API failure independently.
- Measure approximate speed, obstacle response distance, command delivery ratio and run time.
- Prepare labelled subsystem blocks and a one-page wiring diagram.
- Rehearse the demo and keep a known-good firmware build backed up.

Suggested team split

Person	Ownership	Deliverable
Member A	Rover firmware, motors, safety state machine.	Reliable manual and autonomous drive.
Member B	LoRa link, remote base and packet tests.	Command/telemetry demo and link stats.
Member C	ESP32-CAM stream and gateway/dashboard.	Live video, dashboard and event log.
Member D (if present)	AI integration, integration testing, slides/demo.	Mission JSON, image analysis and final demo script.

11. Test Plan and Acceptance Criteria

Treat the targets below as proposed test goals, not measured results. Record actual values during validation, including the test environment, battery condition and radio settings.

Test ID	Area	Procedure	Pass criteria
T-01	Motor control	Test forward/reverse/turn at low speed with wheels lifted.	Correct direction; no resets; stop disables drive.
T-02	Physical stop	Press stop during motion in every mode.	Motor drive disabled immediately by hardware path.
T-03	Ultrasonic safety	Approach a large flat obstacle slowly, then at demo speed.	Rover halts before contact with margin; results recorded.
T-04	Sensor invalidity	Disconnect ultrasonic or return invalid/no echo.	Forward movement inhibited or rover enters safe state.
T-05	LoRa packet delivery	Send numbered test messages at planned distances.	Compute received/expected ratio and note environment.
T-06	Stale command	Stop sending valid drive commands during motion.	Drive command expires and rover stops within configured timeout.
T-07	Duplicate/out-of-order	Replay old packet and duplicate sequence number.	Duplicate/stale command is ignored.
T-08	Camera path loss	Disconnect Wi-Fi while streaming.	Video fails gracefully; rover does not continue blind on AI assumption.
T-09	AI API unavailable	Disable internet or invalidate test key.	Local obstacle logic/stop still works; UI indicates AI unavailable.
T-10	AI invalid output	Return unknown action or out-of-range speed/duration.	Gateway/firmware rejects it; logs the rejection.
T-11	Power transient	Start/stop motors repeatedly while monitoring MCU/radio.	No unexplained reset; if reset occurs, improve power/ground design.

Test ID	Area	Procedure	Pass criteria
T-12	Battery endurance	Run a repeatable mixed-mode route.	Record elapsed time and battery voltage/current if measured.

Useful metrics for final PPT

- Obstacle stopping distance at a stated speed and surface (record measured values).
- LoRa packet delivery ratio and approximate reliable range in the actual test environment.
- Manual command-to-motion latency and fail-safe stop time measured during testing.
- Camera frame rate/resolution actually achieved; AI inspection turnaround time actually observed.
- Runtime under a documented battery and payload configuration.

12. Risks, Limitations and Mitigations

Risk	Why it matters	Mitigation
AI hallucination or wrong object judgement	AI can misinterpret an image or return a plausible but wrong action.	AI proposes only high-level actions; use confidence/recheck logic, human confirmation for sensitive tasks, and deterministic safety interlocks.
Wi-Fi video unavailable at range	LoRa cannot transport the camera stream.	Local autonomous obstacle avoidance continues; show camera/AI as unavailable until Wi-Fi path returns.
LoRa interference or packet loss	RF conditions depend on antenna, band, power, placement and obstacles.	ACKs, sequence IDs, bounded retries, heartbeat and stop-on-stale-command.
Power brownout / electrical noise	Motors draw large startup/stall currents and can reset logic boards.	Separate motor and logic rails, adequate regulators, short high-current paths, bulk capacitance and common ground as needed.
Incorrect GPIO mapping	ESP32-CAM and ESP32 variants have different pin restrictions.	Wait for exact board/module names, avoid reserved camera/flash/boot pins, test one interface at a time.
Radar misuse	Presence sensor interpreted as rangefinder or person identity detector.	Use module documentation; radar event is an optional cue, ultrasonic is primary near obstacle range.
No reliable position estimate	IMU drift and wheel slip accumulate; a compass may be disturbed by motor currents.	Describe navigation as reactive; add encoders and localization later.
Radio compliance	Frequency and equipment rules vary by region and device category.	Verify the exact frequency/band, allowed RF parameters and applicable WPC/DoT requirements before field use.
Credential/privacy exposure	API key or camera images accidentally exposed.	Keep keys on gateway; use access controls; minimize retained images; avoid public exposure of the dashboard.

Known limitations of version 1

- No reliable SLAM/map-based navigation unless additional sensors/odometry and a localization approach are developed.
- Object recognition depends on image quality, viewpoint, lighting, network availability and AI API latency.
- DHT11 has slow/limited ambient sensing and should not be treated as an industrial safety instrument.
- MPU9250 alone does not give absolute position. GPS is useful primarily outdoors with adequate satellite visibility.
- A generic development prototype is not certified for life-critical, industrial, or hazardous-area use.

13. Presentation Outline (10 Slides)

Slide	Recommended content
1. Title	AURA-X: AI-Powered Exploration Rover. One-sentence pitch, team, event.
2. Problem	Remote inspection is difficult in unfamiliar or obstructed spaces; connectivity may be inconsistent.
3. Proposed solution	Manual + local autonomous + AI-assisted missions in a modular rover.
4. System architecture	Show ESP32 main controller, ESP32-CAM, sensors, LoRa base and laptop AI gateway.
5. Hardware stack	Show actual rover photo and table of parts already owned vs parts still required.
6. Dual communications	Explain LoRa packets for telemetry/commands and Wi-Fi for images; explicitly state they do different jobs.
7. AI workflow	Natural-language intent -> schema -> camera image analysis -> validated bounded action -> local execution.
8. Safety and fail-safe	Physical stop, distance guard, command lease, local autonomy, API/link-loss handling.
9. Live demo and measurements	Demonstrate manual drive, obstacle avoidance, LoRa telemetry, one AI inspection. Use actual measured metrics only.
10. Impact and future scope	Search & rescue support, industrial inspection, agriculture/warehouse exploration; encoders, mapping, improved CV as future work.

30-second project pitch

PITCH

“AURA-X is a multi-mode exploration rover that combines local obstacle avoidance, camera-based perception, and a LoRa link for long-range commands and telemetry. A laptop gateway uses AI to interpret missions and analyze selected camera frames, while the onboard ESP32 retains control of movement and safety. The rover can therefore keep its basic safety and navigation behaviours running even when cloud AI or Wi-Fi video is unavailable.”

Poster-ready differentiators

- Two distinct radio paths used for different jobs, not a claim that LoRa carries video.
- AI high-level reasoning bounded by deterministic embedded safety logic.
- Failure-mode demonstrations: not just normal operation.
- Measured communication and stopping performance rather than unsupported range or “accuracy” claims.

14. Final Build Checklist

Inventory and identification

- ☐ Confirm exact ESP32 board model and ESP32-CAM variant.
- ☐ Read the markings on both LoRa modules and identify band/frequency.
- ☐ Identify mmWave radar model and interface voltage.
- ☐ Identify motor driver and motor rated/stall current.
- ☐ Confirm battery chemistry, cell count, voltage and safe charging method.

Hardware bring-up

- ☐ Motor driver tested with wheels lifted.
- ☐ Main ESP32 and ESP32-CAM each boot reliably.
- ☐ Ultrasonic ECHO is level-shifted before ESP32 GPIO.
- ☐ LoRa powered from suitable 3.3 V rail with antenna installed.
- ☐ Emergency stop physically disables motor driver.

Firmware and links

- ☐ Command parser has strict range checks and known enum values.
- ☐ Duplicate and stale commands rejected.
- ☐ LoRa heartbeat and telemetry visible at base.
- ☐ Mode transitions and safe-stop path tested.
- ☐ Camera stream works on the intended Wi-Fi network.

AI / dashboard

- ☐ API key stored outside source code and browser bundle.
- ☐ AI output constrained to known schema/actions.
- ☐ Every proposed action re-validated before motion.
- ☐ AI/API timeout and internet loss shown in UI.
- ☐ Event log shows sensor evidence and action.

Final demo

- ☐ Charge battery and bring spare cables / connectors.
- ☐ Bring backup firmware and known-good test sketch.
- ☐ Keep an offline demo path prepared.
- ☐ Clearly label illustrative vs measured results.
- ☐ Have a physical stop accessible to the operator.

Decision list to finalize before exact wiring

Need to confirm	Why it changes the design
Exact LoRa module and frequency	Determines wiring, library configuration, antenna and compliant RF settings.
Exact motor driver + motor specs	Determines whether it can safely drive the chosen 2WD/4WD motors.
ESP32-CAM variant	Determines camera pins, available GPIOs and viable serial connections.
Exact mmWave model	Determines power/interface and whether it reports presence, motion zones or richer data.
Battery voltage and capacity	Determines motor suitability, regulator choice and estimated runtime.

15. Technical References

Use these authoritative resources to confirm the final implementation. Datasheets and board variants change; check the specific module revision used in the build.

[1] Espressif Systems, ESP32 Series Datasheet. Open source
Use for ESP32 capabilities, power, peripheral interfaces and pin caveats.

[2] Espressif Systems, ESP-IDF GPIO & RTC GPIO documentation. Open source
Use to check GPIO restrictions and interface configuration; exact board design still matters.

[3] Espressif Systems, ESP-IDF Watchdogs documentation. Open source
Background for watchdogs and avoiding blocked tasks in embedded firmware.

[4] Semtech, SX1276 LoRa Connect product page. Open source
Example SX1276 family reference; verify exact radio IC and module before adopting settings.

[5] OpenAI API documentation, Images and Vision. Open source
Reference for sending image inputs for model analysis.

[6] OpenAI API documentation, Structured Outputs. Open source
Reference for schema-constrained model output; application validation remains necessary.

[7] Government of India, Department of Telecommunications, Equipment Type Approval (ETA). Open source
Official overview of WPC/DoT ETA. Verify applicability, permitted band and conditions for the specific radio/product.

Closing recommendation

BUILD THE CORE FIRST

First get four things working end-to-end: motor control with a physical stop, ultrasonic obstacle stop, LoRa command/telemetry between two nodes, and a camera stream on Wi-Fi. Only then add AI mission interpretation. This order provides a demonstrable rover even if the advanced feature takes longer than expected.